AI-Native Product Benchmark — Grounded in Our Own Data

AI 原生产品基准研究——以我们自己的数据为依据

Eight YC-backed companies (Marble, qomplement, Whitespace, Corvera, Handled, Airgoods, Vori, Haladir) compared on product architecture, workflow philosophy, data model, AI role and operating model. Every important idea is tied to a problem actually observed in The Daily Centro's StoreHub data (snapshot 2026-10-08). Ends with one comparison matrix, a warehouse process-truth map, nine design principles and a prioritized V2.1 implementation table for Retail OS V2.1. Company information is from public self-reported sources (YC pages, launch posts, company sites, job postings) — treat their performance claims as directional, not verified. Our internal numbers are queried live from the RetailOS database.

对照研究八家 YC 支持的 AI 原生公司（Marble、qomplement、Whitespace、Corvera、Handled、Airgoods、Vori、Haladir），比较维度为产品架构、工作流哲学、数据模型、AI 角色与运营模式。每个重要观点都对应一个在 The Daily Centro 的 StoreHub 数据中真实观察到的问题（数据快照 2026-10-08）。文末附一张对比矩阵、一张仓库流程真相图、九条 Retail OS V2.1 设计原则和一张 V2.1 优先级实施表。公司信息均来自公开的自我介绍渠道（YC 页面、发布帖、官网、招聘帖），其性能数据请视为方向性参考；我方内部数字均实时查询自 RetailOS 数据库。

---

1 • What our own data shows / 我们自己的数据揭示了什么

These are the observed problems used throughout this document. They come from the live RetailOS database (1,440 products in the catalog, of which 1,425 are stocked at the 4 tracked locations = 5,700 product×location stock records; 63 days of sales; 42 suppliers).

以下是本文反复引用的实际观察到的问题，全部来自 RetailOS 生产数据库（商品目录 1,440 个，其中 1,425 个在 4 个有库存记录的地点在营 = 5,700 条“商品×地点”库存记录；63 天销售数据；42 家供应商）。

#	Observed problem 观察到的问题	The numbers 数据
P1	Inventory records are wrong at scale. 库存记录大面积失真。	739 of the 5,700 stock records (1,425 stocked products × 4 tracked locations: Bantayan, Getafe, Tabunoc, Warehouse) are negative (13%). That is 574 distinct products — 40% of everything we stock — negative at at least one location; worst at Bantayan (400 rows) and Tabunoc (314), while Getafe is clean. E.g. Baygon Mosq Coil shows −540 at the warehouse. And 64% of all stock records are ≤ 0 right now (741 negative + 2,907 zero of 5,700). 5,700 条库存记录（1,425 个在营商品 × 4 个有库存记录的地点：Bantayan、Getafe、Tabunoc、仓库）中有 739 条为负（13%）。涉及 574 个不同商品——占全部在营商品的 40%——至少在一个地点为负；Bantayan 最严重（400 条），Tabunoc 次之（314 条），Getafe 干净。例如仓库的 Baygon 蚊香显示 −540。目前 64% 的库存记录 ≤ 0（5,700 条中 741 条为负、2,907 条为零）。
P2	Replenishment spend is blocked by untrusted data. 补货建议因数据不可信而被卡住。	731 suggestions sit in needs_review worth ₱51,216, versus only 66 clean pending ones worth ₱7,315. Blocking reasons: negative_stock 1,166×, no_supplier 232×, conflicting_po 46×. 731 条建议处于待审查状态、涉及 ₱51,216，而干净的待处理建议仅 66 条、₱7,315。阻塞原因：负库存 1,166 次、无供应商 232 次、采购单冲突 46 次。
P3	Supplier and product master data is incomplete. 供应商与商品主数据不完整。	371 of 1,440 products (26%) have no supplier; 60 have no barcode; 137 POs are open and 25 were flagged stale — so “incoming stock” cannot be trusted. 1,440 个商品中 371 个（26%）没有供应商；60 个没有条码；137 张采购单未关闭、其中 25 张被标记为滞留——因此“在途库存”不可信。

#	Observed problem 观察到的问题	The numbers 数据
P4	Sales history is shallow. 销售历史太短。	63 days of synced sales (2026-08-25 → 2026-10-08). No seasonal baseline exists; StoreHub keeps no inventory history, so RetailOS snapshots are the only record. 仅同步了 63 天销售（2026-08-25 至 2026-10-08）。没有季节性基线；StoreHub 不保存库存历史，RetailOS 的快照是唯一记录。
P5	The AI provider is a reliability risk. AI 服务商是可靠性风险。	5 of 20 AI calls failed: deprecated model (HTTP 404), quota exceeded (429), provider overload (503). Operations must never depend on the AI being available. 20 次 AI 调用失败 5 次：模型下线（404）、配额超限（429）、服务商过载（503）。业务运营绝不能依赖 AI 的可用性。
P6	Process features exist but adoption lags. 流程功能已上线但使用率低。	3 inspections missed and 11 pending; the expiry module has zero batches recorded; no cash closes submitted yet. 3 次巡检漏做、11 次待办；效期模块零记录；尚无一条现金结账记录。

Metric validation (requested 2026-10-08): the “619 phantom stockouts” figure was re-checked and redefined. The original number compared today's inventory with up to 44-day-old sales — it cannot prove a stockout at the moment of sale, because daily stock snapshots exist only since 2026-10-07 (StoreHub keeps no inventory history; RetailOS began snapshotting on start-up). Recomputed on the same day it reads 621, which confirms the drift. The snapshot-verified version — the only defensible one — is: on 2026-10-07, 134 of 227 sold product-days (59%) show ≤ 0 on hand in that same day's own snapshot — 125 distinct products in a single day. The historical rate cannot be reconstructed; from 2026-10-07 onward the daily snapshot makes it measurable, and it should be tracked as a KPI.

指标核验（2026-10-08 应要求复核）：“619 个幽灵缺货”这一数字已重新校验并重新定义。原数字用今天的库存去对比最早 44 天前的销售，无法证明销售发生时点缺货——因为每日库存快照自 2026-10-07 才存在（StoreHub 不保存库存历史，RetailOS 从启动之日起才开始快照）。按原口径当日重算为 621，本身也说明数据在漂移。经得起检验的口径只有一个：2026-10-07 当天，227 个有销量的“商品×日”中有 134 个（59%）在当日快照中库存 ≤ 0——仅一天就涉及 125 个不同商品。历史比率已无法还原；自 2026-10-07 起，每日快照使该指标可测量，应作为 KPI 持续跟踪。

2 • Warehouse process truth — where bad data is born / 仓库流程真相——脏数据在哪里产生

The real central-warehouse flow, step by step: what actually happens, who does it, which tool is used, when the data is recorded, and where a wrong record can be created. Grounded in the code and the database: the receiving module has 0 records ever, the dispatch/confirmation flow has 0 batches, cycle counts: 1 (still open), expiry batches: 0, cash closes: 0; inspections: 14 (3 missed). The physical-confirmation layer exists but is unused — so almost every inventory-changing event leaves a trace only as an unexplained polling diff (283 diffs observed since 2026-10-07, with no actor and no reason).

中央仓库的真实流程，逐步列出：实际发生了什么、谁在做、用什么工具、数据何时记录、错误记录在哪里产生。依据代码与数据库：收货模块从未有过一条记录、发货/确认流程从未有过一个批次、循环盘点仅 1 次（仍未完成）、效期批次 0、现金结账 0；巡检 14 次（3 次漏做）。实物确认层存在但未被使用——因此几乎每一个改变库存的事件，只留下一条无法解释的轮询差异（自 2026-10-07 观察到 283 条差异，没有操作人、没有原因）。

# • Step 步骤	What really happens today 真实现状	Who • tool 谁 • 工具	When recorded 何时记录	Where bad data is born 脏数据产地
1 · Supplier receiving 供应商收货	Goods arrive at the warehouse; quantities are keyed against the PO in StoreHub. The RetailOS counted-receiving module exists but has never been used (0 records). 货物到仓，数量由人工在 StoreHub 按采购单录入。RetailOS 的收货点验模块存在但从未使用（0 条记录）。	Warehouse receiver · StoreHub back office + paper invoice 仓库收货员 · StoreHub 后台+纸质发票	When keyed — sometimes days after the goods arrived. 录入时——有时在货到数日之后。	Keyed ≠ counted; partial receipts invisible; PO left open (25 stale POs poison "incoming stock"). 录入≠点验；部分到货不可见；采购单不关闭（25 张滞留单污染"在途库存"）。
2 · Receiving inspection 收货质检	A Warehouse Inspection checklist exists (due 12:00 daily); 14 inspections done, 3 missed. No photo evidence is attached to a delivery. 有仓库巡检清单（每日 12:00 截止）；已做 14 次、漏做 3 次。收货不附照片证据。	Warehouse checker · RetailOS phone form 仓库检查员 · RetailOS 手机表单	At checklist submit — if it happens. 提交清单时——如果做了的话。	Skipped or pencil-whipped checks; damaged cases accepted unnoticed. 检查被跳过或敷衍；破损箱被悄悄收下。
3 · Putaway 上架	No system support at all: no bin locations in StoreHub or RetailOS. Goods go where the picker remembers. 完全没有系统支持：StoreHub 与 RetailOS 都没有库位。货放在拣货员记得的地方。	Warehouse staff · memory / none 仓库员工 · 凭记忆/无工具	Never. 从不记录。	Goods physically misplaced; later "not found" → emergency adjustment. 实物放错位置；日后"找不到"→ 临时调整。
4 · Store request 门店要货	The RetailOS engine suggests transfers deterministically; stores also ask informally by chat/phone. RetailOS 引擎确定性地建议调拨；门店也通过聊天/电话口头要货。	Engine + store manager · RetailOS / Lark chat 引擎+门店经理 · RetailOS / Lark 群	Suggestions daily; informal requests never. 建议每日生成；口头要货从不记录。	Informal demand untracked; engine inputs poisoned by negative stock (P1/P2). 口头需求不留痕；引擎输入被负库存污染（P1/P2）。
5 · Picking 拣货	The picker follows the keyed transfer; no scan verification of item or quantity. 拣货员按已录入的调拨单拣货；商品与数量均无扫码核验。	Warehouse picker · StoreHub transfer + paper 仓库拣货员 · StoreHub 调拨单+纸	When the transfer is keyed. 录入调拨单时。	Wrong item/qty picked — undetected until the store counts, if ever. 拣错商品/数量——要等门店盘点才发现，如果门店盘点的话。
6 · Dispatch 发货	RetailOS has a dispatch confirmation step (FIFO) — 0 batches have ever used it. Stock leaves when the StoreHub transfer is keyed. RetailOS 有发货确认环节（先进先出）——但从未有一个批次使用过。StoreHub 调拨单一录，货即出仓。	Warehouse lead · StoreHub (+ unused RetailOS step) 仓库主管 · StoreHub（RetailOS 环节闲置）	At transfer keying, before or after the physical move. 录单时，早于或晚于实物移动。	Keying and physical move out of sync; timing gaps read as discrepancies. 录单与实物移动不同步；时间差被读成差异。
7 · In transit 在途	No tracking between dispatch and store receipt. 发货与门店收货之间没有任何跟踪。	Driver · none 司机 · 无工具	Never. 从不记录。	Loss, theft and damage invisible → later becomes an unexplained difference. 丢失、偷盗、破损不可见——日后变成无法解释的差异。
8 · Store receiving 门店收货	The store should count against the transfer; the RetailOS confirmation field exists but is unused. 门店应按调拨单点验；RetailOS 有收货确认字段但无人使用。	Store staff · StoreHub auto-increment 门店员工 · StoreHub 自动入库	Automatically when the transfer completes — counted or not. 调拨完成即自动入库——无论是否点验。	Store accepts without counting; the difference is absorbed silently into stock. 门店不点数就收货；差异被悄悄吞进库存。
9 · Differences 差异	RetailOS DQ rules detect some of it after the fact (PO received but stock never moved, etc.); only critical findings open an issue. RetailOS 的 DQ 规则事后能发现一部分（如采购单已收货但库存未动）；只有严重项才会生成工单。	System · RetailOS DQ engine 系统 · RetailOS DQ 引擎	Hours to days after the event. 事件发生后数小时至数天。	Unexplained deltas accumulate → negative stock (741 rows). 无法解释的差异不断累积 → 负库存（741 行）。

# • Step 步骤	What really happens today 真实现状	Who • tool 谁 • 工具	When recorded 何时记录	Where bad data is born 脏数据产地
10 · Returns 退货	Handled in the StoreHub back office; no RetailOS trace. 在 StoreHub 后台处理；RetailOS 无记录。	Store/warehouse staff · StoreHub 门店/仓库员工 · StoreHub	When keyed. 录入时。	Returns keyed to the wrong location or product. 退货错录地点或商品。
11 · Damages / write-offs 报损/核销	The AI's write-off tool can only instruct "use the StoreHub back office" — no reason, no photo, no actor captured. This is the likely birthplace of negative stock. AI 的核销工具只能提示"请到 StoreHub 后台操作"——不记录原因、照片、操作人。这里很可能就是负库存的出生地。	Floor staff · StoreHub or nowhere 一线员工 · StoreHub 或不记录	When someone remembers to key it. 有人记得去录的时候。	Damage thrown away unkeyed, or keyed twice. 报损扔掉不录，或重复录入。
12 · Cycle count 循环盘点	The counts module exists; exactly 1 count has ever been created — still open. 盘点模块存在；但总共只创建过 1 次盘点——至今未完成。	Store/warehouse staff · RetailOS phone form 门店/仓库员工 · RetailOS 手机表单	On submit — has not happened yet. 提交时——尚未发生过。	Counts not done → records drift; or bulk adjustments without per-item evidence. 不盘点→记录漂移；或整批调整而无单品证据。
13 · Stock adjustment 库存调整	Done directly in StoreHub; RetailOS sees only a polling diff (283 since 2026-10-07) with no actor and no reason. 直接在 StoreHub 调整；RetailOS 只能通过轮询看到差异（10-07 以来 283 条），没有操作人、没有原因。	Anyone with StoreHub access · StoreHub 任何有 StoreHub 权限的人 · StoreHub	At keying; visible to us only at the next poll. 录入时；我们要等到下次轮询才看见。	Adjustments without reason = unauditable history. 没有理由的调整 = 无法审计的历史。

Reading the map: of the 13 steps, only sales (POS) and polling diffs leave an automatic trace today. Every physical confirmation step — receiving count, dispatch, store count, write-off reason — exists in RetailOS but is unused. That is the operational root cause behind P1 and P2, and the reason for principle 9.

如何读这张图：13 个环节中，今天只有销售（POS）和轮询差异会自动留痕。每一个实物确认环节——收货点验、发货、门店点收、报损原因——在 RetailOS 里都存在但无人使用。这就是 P1、P2 背后的运营根因，也是第 9 条原则存在的理由。

3 • The eight companies — corrected identities / 八家公司——更正后的真实身份

Several names on the study list are not what they suggest; this matters for what can fairly be learned from each. Five of the eight pivoted or rebranded (Marble←Truffle, qomplement mid-pivot, Handled←Zimi, Airgoods rebrand, Corvera repositioned three times in nine months). Only Vori and Haladir have held one thesis.

研究清单上的几个名字与其真实业务并不相符——这决定了从每家公司身上能公允地学到什么。八家中有五家经历过转型或更名（Marble 前身 Truffle、qomplement 正在转型、Handled 前身 Zimi、Airgoods 更名重塑、Corvera 九个月内三次调整定位）。只有 Vori 和 Haladir 始终坚持同一主线。

Company 公司	What it actually is 实际情况	Why it matters to us 与我们的关系
Marble (YC S26)	Restaurant back-of-house: phone camera + voice inventory capture, invoice OCR, agents draft orders for manager approval. 餐厅后厨：手机摄像头+语音盘点采集、发票 OCR，代理起草订单、经理审批。	Capture-first: proves the moat is manufacturing ground truth where none exists. 采集优先：证明护城河是在没有数据的地方制造数据。

Company 公司	What it actually is 实际情况	Why it matters to us 与我们的关系
qomplement (YC P26)	Document-parsing workflows: OCR+LLM extraction into deterministic trigger→transform→act pipelines. 文档解析工作流：OCR+LLM 提取，接入确定性的触发→转换→执行管道。	Clearest published deterministic/AI split; template as the AI's context contract. 公开的确定性/AI 分工最清晰；模板即 AI 的上下文契约。
Whitespace (YC S26)	Wholesale-distribution overlay: reads ERP + inboxes, enters orders, explains every number with worked decompositions. 批发分销叠加层：读取 ERP+收件箱、录入订单，每个数字都有演算分解。	Same overlay archetype as RetailOS; corrections become the next draft's context. 与 RetailOS 同为叠加层架构；人的修正会成为下次草稿的上下文。
Corvera (YC W26)	Ops agents for CPG brands; "context layer" exposed via MCP; human-in-the-loop approvals, full audit log claimed. 面向 CPG 品牌的运营代理；通过 MCP 暴露"上下文层"；人工审批，自称有完整审计日志。	Overlay + approval + audit claims match our philosophy (unverified). 叠加层+审批+审计的主张与我们理念一致（未经验证）。
Handled (YC S24)	Post-order ops agent for e-commerce/CPG brands; LLMs wrapped in deterministic rules; graduated autonomy, one-tap approval in Slack. 面向电商/CPG 品牌的订单后运营代理；LLM 包裹在确定性规则内；渐进自治，Slack 一键审批。	"Follows rules you set, instead of hallucinating"; approvals where managers live. "遵循你设定的规则，而不是产生幻觉"；在经理常驻的地方审批。
Airgoods (YC S23)	Wholesale marketplace for specialty food; POS-sync reorder center with deterministic threshold rules; retailer always confirms orders. 特色食品批发交易市场；基于 POS 同步的补货中心使用确定性阈值规则；零售商始终确认订单。	Independently converged on our pattern: read-only POS sync, snapshots, freshness warnings, no auto-ordering. 独立地走到了与我们相同的模式：只读 POS 同步、快照、数据新鲜度警告、不自动下单。
Vori (YC W20)	Full POS replacement (VoriOS) for US supermarkets; perpetual-inventory ledger; approval gate on price changes; $32M raised. 面向美国超市的完整 POS 替换（VoriOS）；永续库存台账；调价需审批；融资 3200 万美元。	The inverse of our architecture — shows the ceiling an overlay inherits, and the cost of owning the POS. 与我们架构相反——展示了叠加层会继承的天花板，以及拥有 POS 的代价。
Haladir (YC W26)	Logistics decision layer over WMS/TMS/ERP; deterministic solvers decide, LLM formalizes rules and explains; decision traces; review-mode-first autonomy. 架在 WMS/TMS/ERP 之上的物流决策层；确定性求解器做决策，LLM 负责规则形式化与解释；决策轨迹；自治从审查模式起步。	Best-documented architecture of the eight; refusal semantics and backtest discipline worth borrowing. 八家中架构文档最完整；其拒绝语义与回测纪律值得借鉴。

4 • The important ideas, each tied to an observed problem / 重要观点——每条都对应一个已观察到的问题

Format per idea, as briefed: Warehouse problem → Current process → Competitor approach → What we can learn → Proposed Retail OS approach.

按要求的格式：仓库问题 → 现状流程 → 竞品做法 → 可借鉴之处 → Retail OS 建议方案。

Idea 1 • Inventory records lie — fix at capture time, not report time / 库存记录会撒谎——要在采集环节修正，而不是在报表环节

Observed problem **仓库问题**	739 negative stock records (13% of all product×location records) covering 574 distinct products — 40% of the range. Snapshot-verified: on 2026-10-07, 59% of sold product-days show ≤ 0 on hand in that day's own snapshot. StoreHub simply does not know what is on the shelf. (P1) 739 条负库存记录（占全部“商品×地点”记录的 13%），涉及 574 个不同商品——占在营商品的 40%。经快照验证：2026-10-07 当天 59% 有销量的“商品×日”在当日快照中库存 ≤ 0。StoreHub 根本不知道货架上有什么。（P1）
Current process **现状流程**	RetailOS detects the damage after the fact: the DQ engine flags negative_stock (739 open events) and replenishment marks affected lines needs_review. Nothing prevents the bad record from being born. RetailOS 只能事后发现：DQ 引擎标记负库存（739 条未结事件），补货模块把受影响行标记为待审查。但无法阻止错误记录的产生。
Competitor approach **竞品做法**	Marble manufactures ground truth at the source (camera/voice counts as staff move through the kitchen). Vori goes further: it owns the ledger — every sale, invoice and floor write-off with reason codes flows into one perpetual-inventory record. Marble 在源头制造事实（员工在厨房走动时用摄像头/语音盘点）。Vori 更进一步：自己拥有台账——每笔销售、每张发票、每次带原因代码的现场报损都流入同一本永续库存账。
What we can learn **可借鉴之处**	An overlay inherits the POS's data-quality ceiling. Detection is necessary but not sufficient; the cheapest place to fix a wrong record is the moment it is created, by a human holding a phone. 叠加层会继承 POS 的数据质量天花板。检测是必要的但并不够；修正错误记录成本最低的时机，就是它产生的那一刻——由拿着手机的人当场完成。
Proposed Retail OS approach **Retail OS 建议方案**	Stay an overlay (Vori's path needs $32M), but move correction upstream: targeted cycle counts auto-generated from the 739 negative rows (worst first), receiving variance captured on-phone at the dock, and a DQ trend per product so we see whether the record is improving, not just wrong today. 坚持叠加层路线（Vori 的路线需要 3200 万美元），但把修正动作前移：由 739 行负库存自动生成定向循环盘点（最差优先）、收货时在码头用手机记录差异、为每个商品建立 DQ 趋势，让我们看到记录是在改善而不仅是今天错了。

Idea 2 • Refuse loudly — a blocked suggestion must name its missing data / 响亮地拒绝——被卡住的建议必须指明缺什么数据

Observed problem **仓库问题**	₱51,216 of suggested order value sits in needs_review — 7× the clean pending value (₱7,315). The largest blockers are negative_stock (1,166) and no_supplier (232). (P2, P3) 价值 ₱51,216 的订货建议处于待审查状态——是干净待处理值（₱7,315）的 7 倍。最大的阻塞原因是负库存（1,166 次）和无供应商（232 次）。（P2、P3）
Current process **现状流程**	Lines are quietly flagged needs_review with a reasons list inside the “Why this number” drill-down. The manager has to open each line to discover the suggestion cannot be trusted. 各行被悄悄标记为待审查，原因列表藏在“为什么是这个数”的下拉里。经理必须逐行点开才能发现建议不可信。
Competitor approach **竞品做法**	Haladir's solvers refuse infeasible plans and state the reason; nothing is applied silently. Their POC must reproduce the customer's measured history before any recommendation counts. Haladir 的求解器会拒绝不可行方案并说明原因，绝不静默执行。其概念验证必须能复现客户的实测历史数据，建议才会被采信。
What we can learn **可借鉴之处**	A refusal is more useful than a caveat. ₱51k of silent needs_review is really a data-repair work queue wearing a replenishment costume. 拒绝比附注更有用。₱51k 静默的待审查实际上是一个伪装成补货问题的数据修复工单队列。

Proposed Retail OS approach **Retail OS 建议方案**	Promote needs_review to first-class refusal cards: “Not suggesting an order for X — stock record is −540, count it” with a direct link to the fixing screen (cycle count, supplier assignment, PO close-out). Measure success as ₱-value unblocked per week, not suggestions generated. 把待审查升级为一等的拒绝卡片：“不为 X 建议订货——库存记录为 −540，请盘点”，并附上直达修复界面的链接（循环盘点、供应商指派、采购单关闭）。成功指标改为每周解锁的金额，而不是生成的建议数。

Idea 3 • Every recommendation must be replayable / 每条建议都必须可复算

Observed problem **仓库问题**	Only 63 days of sales history exist, and phantom stockouts (59% of sold product-days showed ≤ 0 stock in the 2026-10-07 snapshot) bias the average-daily-sales that drives every suggestion. (P4, P1) 销售历史只有 63 天，而幽灵缺货（2026-10-07 快照中 59% 有销量的“商品×日”库存 ≤ 0）会使驱动每条建议的日均销量产生偏差。（P4、P1）
Current process **现状流程**	The V1 formula is deterministic and shows its working per line, and it was cross-checked once by hand (191/191 products matched) — but there is no harness that re-proves it after every change. V1 公式是确定性的并逐行展示计算过程，也曾手工交叉核对过一次（191/191 个产品一致）——但没有在每次变更后重新验证的机制。
Competitor approach **竞品做法**	Haladir: same inputs must produce the same plan, every plan carries a decision trace, and validation against historical data is a gate, not a courtesy. Whitespace shows worked decompositions for every number. Haladir：相同输入必须得到相同方案，每个方案都带决策轨迹，且用历史数据验证是一道门槛而不是礼节。Whitespace 为每个数字展示演算分解。
What we can learn **可借鉴之处**	Trust in a formula is earned repeatedly, not once. With 63 days of noisy data, the formula will be tuned — each tuning needs a machine that says “yesterday's answers would still be right.” 对公式的信任需要反复赢得，而不是一次到位。只有 63 天带噪声的数据，公式必然会被调优——每次调优都需要一台机器来回答“昨天的结论放到今天仍然成立吗”。
Proposed Retail OS approach **Retail OS 建议方案**	Build a backtest harness: replay the formula against past snapshots, compare suggested vs actual outcomes (we already store recommendation outcomes), version the formula, and block deploys that regress acceptance or stockout rates on replay. 构建回测工具：用历史快照重放公式，对比建议值与实际结果（我们已存储建议结果数据），对公式做版本管理，并阻止在重放中导致接受率或缺货率倒退的变更上线。

Idea 4 • Context is a contract — and corrections are the best context / 上下文是一种契约——而人的修正是最好的上下文

Observed problem **仓库问题**	371 products (26%) have no supplier, so warehouse ordering cannot group demand by vendor; 60 products have no barcode, slowing receiving and counts. Meanwhile managers adjust suggested quantities with reasons we record but never reuse. (P3) 371 个商品（26%）没有供应商，仓库订货无法按供应商聚合需求；60 个商品没有条码，拖慢收货与盘点。同时，经理修改建议数量时填写的理由被记录了，却从未被再次利用。（P3）
Current process **现状流程**	The AI receives a numbered-facts context package built by code; replenishment outcomes (accept/modify/reject + reason) are stored for metrics only. AI 接收由代码构建的编号事实上下文包；补货结果（接受/修改/拒绝+理由）仅用于统计指标。
Competitor approach **竞品做法**	qomplement's docs: extraction quality is bounded by an explicit template — the contract is the context. Whitespace's “living model”: “change a draft once and the next one comes out right”; business rules and per-person corrections are stored and read back by every agent. qomplement 的文档：提取质量受限于一份显式模板——契约即上下文。Whitespace 的“活体模型”：“改一次草稿，下次就对了”；业务规则和每个人的修正都被存储并被每个代理读取。

What we can learn **可借鉴之处**	The context schema pays more than prompt tuning. And the highest-value context is not more data — it is the recorded judgment of the people who already know the operation. 投资于上下文结构设计比调提示词回报更高。而最高价值的上下文不是更多数据——而是那些最懂运营的人留下的、被记录下来的判断。
Proposed Retail OS approach **Retail OS 建议方案**	Two moves: (a) a supplier-master cleanup sprint driven by the 371-product list (it directly unblocks ₱51k of Idea 2); (b) feed modification patterns back as facts — "Manager rounded supplier X down 3 times" becomes a cited input to the next suggestion's explanation. 两步走：(a) 按 371 个商品的清单做一次供应商主数据清理冲刺（这直接为观点 2 解锁 ₱51k）；(b) 把修改模式作为事实反馈——"经理第三次下调供应商 X 的数量"成为下一条建议解释中被引用的输入。

Idea 5 • AI is a colleague who can call in sick — operations must not depend on it / AI 是一个会请病假的同事——运营绝不能依赖它

Observed problem **仓库问题**	5 of 20 AI calls failed in production: model deprecated by the provider (404), quota exceeded (429), provider overload (503). (P5) 生产环境 20 次 AI 调用失败 5 次：服务商下线模型（404）、配额超限（429）、服务商过载（503）。（P5）
Current process **现状流程**	Provider chain Anthropic → Gemini → local CLI → off; daily call cap; grounding validation. Briefs fail visibly; nothing operational blocks — by design. 服务商链为 Anthropic → Gemini → 本地 CLI → 关闭；有每日调用上限和落地核查。简报失败会显式可见；任何运营环节都不会因此被阻塞——这是设计使然。
Competitor approach **竞品做法**	Handled wraps LLMs in deterministic rules so the agent "follows your directions, not hallucinates"; graduated autonomy means the deterministic core carries the load and the LLM is interchangeable. Haladir's LLM never computes a decision at all. Handled 把 LLM 包裹在确定性规则中，让代理"遵循你的指示而不是产生幻觉"；渐进自治意味着确定性内核承担负载，LLM 可以替换。Haladir 的 LLM 根本不参与计算决策。
What we can learn **可借鉴之处**	The discipline RetailOS already has is the industry consensus among the serious players. The gap is expectation management: 3 of 5 briefs succeeding still feels broken to a manager at 6am. RetailOS 已有的纪律正是认真玩家的行业共识。差距在于预期管理：5 次简报成功 3 次，在早上 6 点的经理看来仍然是"坏了"。
Proposed Retail OS approach **Retail OS 建议方案**	Keep the boundary; harden the edges: auto-failover model list per provider (the 404 was a deprecated model name), a cached "last good brief" with its timestamp shown, and a public per-provider failure metric on the assistant metrics page so reliability is managed, not discovered. 守住边界，加固边缘：为每个服务商配置自动故障转移的模型列表（404 是因为模型名已下线）、缓存"最近一次成功的简报"并显示其时间戳、在助手指标页公开各服务商的失败率——让可靠性被管理，而不是被偶遇。

Idea 6 • Approvals and capture must live where the people already are / 审批与采集必须发生在人们本来就常驻的地方

Observed problem **仓库问题**	3 inspections missed and 11 pending; the expiry module has zero batches; no cash close has ever been submitted. The processes are built; the floor staff are not in them. (P6) 3 次巡检漏做、11 次待办；效期模块零记录；现金结账从未被提交。流程建好了，但一线员工不在里面。（P6）
Current process **现状流程**	Web app, responsive, with a mobile inspection form (recently redesigned with tap targets and upload tiles); alerts mirror one-way to a Lark chat; approvals exist only inside the app. 响应式 Web 应用，巡检表单已做移动端改造（大按钮、图片上传卡片）；告警单向同步到 Lark 群；但审批只能在应用内完成。

Competitor approach 竞品做法	Handled puts one-tap approval in Slack — “small POs run automatically, big ones wait for your one-tap approval.” Marble's entire capture layer is a phone camera, because that is what kitchen staff already carry. Airgoods confirmed that retailers will not auto-order: the human tap is the product. Handled 把一键审批放进 Slack——“小额采购单自动执行，大额的等你在 Slack 一键批准”。Marble 的整个采集层就是手机摄像头，因为那是后厨员工本来就带在身上的东西。Airgoods 证实零售商不会接受自动下单：人的那一键点击本身就是产品。
What we can learn 可借鉴之处	Adoption follows the path of least resistance. A manager who lives in Lark will approve in Lark; a checker on the floor will capture on a phone or not at all. 采用率遵循阻力最小的路径。常驻 Lark 的经理会在 Lark 里审批；卖场里的员工要么用手机采集，要么干脆不做。
Proposed Retail OS approach Retail OS 建议方案	Close the loop in Lark: alerts already flow one-way — add approve/reject actions there (the spec's deferred “Lark approvals”) for low-risk classes within peso limits, keeping the hash-chained audit entry. For capture: the missed-inspection nudge should arrive in the same chat, with a deep link into the redesigned form. 在 Lark 中闭环：告警已是单向同步——为限额内的低风险类别加上批准/拒绝按钮（即规范中暂缓的“Lark 审批”），同时保留哈希链审计记录。采集侧：漏检提醒应推送到同一个群里，并附上新表单的深度链接。

5 • Final comparison matrix / 最终对比矩阵

Dimensions: archetype · ground truth · integration model · deterministic vs AI · act vs recommend · governance & operating model. Everything below is from public self-reported material; cells are compressed claims, not verified facts.

维度：架构原型 · 事实源 · 集成方式 · 确定性与 AI 分工 · 执行与建议 · 治理与运营模式。以下内容全部来自各公司公开的自我介绍材料；单元格为压缩后的主张，未经核实。

Company	Archetype 原型	Ground truth 事实源	Deterministic / AI 分工	Acts or recommends 执行或建议	Governance & model 治理与模式
Marble	Capture-first overlay 采集优先叠加层	Manufactured: camera/voice counts + invoices + POS 自造：视觉/语音盘点+发票+POS	CV/OCR/docs = AI; forecasting & costing = classical ML (boundary not published) 视觉/文档=AI；预测与成本=传统ML（边界未公开）	Agents draft POs/prep/schedules; managers approve 代理起草采购单/备料/排班；经理审批	“Agents draft, managers approve.” Multi-unit restaurant groups, US. “代理起草，经理审批。”美国多门店餐饮集团。
qomplement	Document-workflow platform 文档工作流平台	Supplier documents (PDF/Excel/email) + existing databases 供应商单据（PDF/Excel/邮件）+既有数据库	Clearest split: deterministic workflow builder; AI only at OCR extraction & field mapping 分工最清晰：确定性工作流构建器；AI 仅做 OCR 提取与字段映射	Claims full execution incl. DB writes “without human input” 自称可全自主执行，含写库“无需人工”	Mid-pivot, contradictory messaging; enterprise PoC sales. 转型中、信息自相矛盾；大客户 PoC 销售。
Whitespace	ERP overlay ERP 叠加层	Existing ERP + inboxes + files 既有 ERP+收件箱+文件	Agents forecast/enter/triage; every number shows a worked decomposition 代理做预测/录入/分拣；每个数字都有演算分解	Acts on order entry & support; recommends on reorders 订单录入与客服自动执行；补货仅建议	Per-workflow autonomy calibration; “living model” of rules+corrections. UK wholesalers. 按工作流校准自治度；“活体模型”存规则与修正。英国批发商。

Company	Archetype 原型	Ground truth 事实源	Deterministic / AI 分工	Acts or recommends 执行或建议	Governance & model 治理与模式
Corvera	Context layer + agents 上下文层+代理	Customer's ERP/portals/syndicated data unified, exposed via MCP 整合客户 ERP/门户/行业数据，经 MCP 暴露	AI parses & matches; workflows execute with HITL approvals (split not detailed) AI 解析与匹配；工作流带人工审批执行（分工未详述）	Does order processing end-to-end; recommends rebalancing 端到端处理订单；调库仅建议	Claims full audit log & access control (unverified). CPG brands >$50M. 自称有完整审计与权限（未验证）。营收>$50M 的 CPG 品牌。
Handled	Ops overlay (ex-Zimi) 运营叠加层（前身 Zimi）	Systems of record + unstructured (3PL emails, sheets, Slack) 记录系统+非结构化数据（物流邮件、表格、Slack）	Explicit: "LLMs with deterministic rules — follows your directions, not its own" 明示："LLM 加确定性规则——遵循你的指示"	Executes: POs, 3PL chasing, invoice audits, month-end close 可执行：采购单、催物流、发票审计、月末结账	Graduated autonomy by amount; one-tap approval in Slack; "everything is logged." 按金额渐进自治；Slack 一键审批；"一切有日志"。
Airgoods	Marketplace 交易市场	Retailer POS snapshots in own Postgres + marketplace data 零售商 POS 快照存入自有 PG + 市场数据	Reorder = deterministic threshold rules, chosen for explainability; no documented LLM agent 补货=确定性阈值规则（为可解释性而选）；无公开的 LLM 代理	Never acts: auto-ordering explicitly out of scope; retailer confirms 绝不自动执行：明示不做自动下单；零售商确认	Free retailer tooling funded by brand commissions; US-only. 零售商免费、品牌佣金补贴；仅限美国。
Vori	POS replacement (VoriOS) POS 替换（VoriOS）	Owns it: perpetual-inventory ledger (sales, invoices, write-offs with reason codes) 自有：永续库存台账（销售、发票、带原因代码的报损）	POS/ledger deterministic; "agents" are rules+statistics with an AI label; OCR/ML in ingestion POS/台账为确定性；"代理"实为贴 AI 标签的规则+统计；采集端用 OCR/ML	Acts on ordering/marketing; price changes need human approval 订货/营销自动；调价需人工审批	FDE-led messy-data onboarding, then own the record. US supermarkets. FDE 主导脏数据接入，然后拥有记录系统。美国超市。
Haladir	Solver decision layer 求解器决策层	WMS/TMS/OMS/ERP unified into an operational graph + process mining 整合 WMS/TMS/OMS/ERP 为运营图+流程挖掘	Strictest: solvers decide (same inputs → same plan); LLM only formalizes rules & explains 最严格：求解器决策（同输入同方案）；LLM 只做规则形式化与解释	Both: Nomos plans; Talon executes with WMS/TMS write-back 两者皆有：Nomos 规划；Talon 执行并回写 WMS/TMS	Decision traces; refuse infeasible with reason; review-mode-first autonomy; backtest gate. 决策轨迹；拒绝不可行方案并说明原因；自治从审查起步；回验门槛。

6 • Design principles for Retail OS V2.1 / Retail OS V2.1 设计原则

Nine principles, each anchored to the observed evidence (P1–P6 from Section 1 and the warehouse map in Section 2) and the competitor that proves or warns against it.

九条原则，每条都锚定在已观察到的证据（第 1 节的 P1–P6 与第 2 节的仓库流程图）以及验证或警示它的竞品之上。

1. Ground truth is captured, not assumed.

People with phones fix records at the moment they are born; reports only find corpses. (P1: 739 negative rows; Marble's capture layer; Vori's ledger.)

1. 事实靠采集，不靠假设。拿着手机的人在记录产生的那一刻修正它；报表只能找到尸体。（P1：739 行负库存；Marble 的采集层；Vori 的台账。）

2. Code calculates; AI explains.

Deterministic formulas decide money; the LLM narrates, answers and proposes — with citations. Never let the model compute an order. (Haladir's solver boundary; our replenishment formula.)

2. 代码负责计算，AI 负责解释。确定性公式决定钱的去向；LLM 只做叙述、答疑和建议——并附引用。绝不让模型计算订单。（Haladir 的求解器边界；我们的补货公式。）

3. Refuse loudly, with the reason.

A blocked suggestion names its missing data and links to the fix. Silent caveats rot trust. (P2: ₱51k in needs_review; Haladir's refusals.)

3. 响亮地拒绝，并说明原因。被卡住的建议要指明缺什么数据并给出修复链接。静默的附注会腐蚀信任。（P2：₱51k 待审查；Haladir 的拒绝机制。）

4. Every recommendation is replayable.

Formula versions + snapshot replay + outcome tracking = trust earned repeatedly. Block changes that regress on replay. (P4: 63 days of history; Haladir's backtest gate; our 191/191 check, made repeatable.)

4. 每条建议都可复算。公式版本+快照重放+结果追踪=反复赢得的信任。重放倒退的变更不得上线。（P4：仅 63 天历史；Haladir 的回验门槛；我们的 191/191 核对应常态化。）

5. Autonomy is earned by decision class, risk, data confidence, reversibility, historical acceptance rate and amount.

A low-value, reversible, well-measured decision class with a proven acceptance rate may run itself; anything novel, irreversible or data-poor waits for a person — and everything lands in the hash-chained audit log. (Handled's graduated gates; our policy engine already checks class, risk, amount, freshness and confidence — wiring in the acceptance rate is V2.1 work.)

5. 自治要按决策类别、风险、数据置信度、可逆性、历史接受率和金额逐步赢得。金额低、可逆、度量充分且接受率已被证明的决策类别可以自动执行；任何新颖、不可逆或数据不足的类别都必须等人——且一切进入哈希链审计日志。（Handled 的渐进闸门；我们的策略引擎已检查类别、风险、金额、数据新鲜度与置信度——接入接受率是 V2.1 的工作。）

6. Corrections are candidate context; outcomes decide whether they become trusted policy.

Record the recommendation, the manager's override, the reason and the actual result — then learn whether the override improved the outcome before letting it influence the next suggestion. An untested correction is an anecdote, not a rule. (P3; Whitespace's living model — but with a proof gate; our outcome tracking makes this measurable.)

6. 人的修正只是候选上下文；结果决定它能否成为可信策略。记录建议、经理的修改、理由和实际结果——先验证修改是否真的改善了结果，再让它影响下一条建议。未经验证的修正只是轶事，不是规则。（P3；Whitespace 的活体模型——但要加一道验证门；我们的结果追踪使这可以度量。）

7. Degrade gracefully; never block operations on AI.

Provider chains fail (P5: 5/20 calls). Cached last-good outputs, visible failure metrics, and an ops loop that is complete with AI off. AI absence is a Tuesday, not an incident.

7. 优雅降级；绝不让运营依赖 AI。服务商链会失败（P5：20 次失败 5 次）。缓存最近一次成功输出、公开失败指标、保证 AI 全关时运营闭环依然完整。AI 缺席应是一个普通的星期二，而不是一次事故。

8. Meet people where they already work.

Approvals in Lark, capture on phones, nudges in chat with deep links. Features that live in a tab nobody opens do not exist. (P6: missed inspections, unused modules; Handled's Slack approvals; Marble's phone-first capture.)

8. 在人们常驻的地方与他们相遇。审批进 Lark，采集用手机，提醒带深度链接进聊天群。藏在无人打开的标签页里的功能等于不存在。（P6：巡检漏做、模块闲置；Handled 的 Slack 审批；Marble 的手机优先采集。）

9. Every inventory-changing real-world event must leave a trace.

Receiving, transfers, returns, damages, write-offs, counts and adjustments must each record an actor, a reason and a quantity, so the system can explain why current inventory has its current value. The warehouse map shows why: today only sales and polling diffs leave an automatic trace — 0 counted receivings, 0 dispatch confirmations and 1 unfinished count ever recorded, while StoreHub adjustments happen with no visible actor. Untraceable events are how 741 negative rows were born.

9. 每一个改变库存的真实事件都必须留下痕迹。收货、调拨、退货、报损、核销、盘点和库存调整都必须记录操作人、原因和数量，系统才能解释当前库存为何是这个数。仓库流程图说明了原因：今天只有销售和轮询差异会自动留痕——点验收货为 0、发货确认为 0、盘点仅有 1 次且未完成，而 StoreHub 里的调整看不到操作人。无法追溯的事件正是 741 行负库存的由来。

7 • V2.1 prioritized implementation table / V2.1 优先级实施表

Every item solves a verified problem from Section 1 or the warehouse map in Section 2 — nothing is here because a YC company has it. Chain per item: Problem → Root cause → Principle → Feature/change, then Data needed · Owner · Acceptance test · KPI. Priority classes: Must build now / Next / Later.

每一项都解决第 1 节或第 2 节仓库流程图中已验证的问题——没有任何一项仅仅因为某家 YC 公司有而被纳入。每项的链条为：问题 → 根因 → 原则 → 功能/改动，随后是所需数据 · 负责人 · 验收测试 · KPI。优先级：立即建设 / 下一步 / 以后。

Item 项目	Problem → root cause → principle → change 问题→根因→原则→改动	Data • owner • acceptance • KPI 数据•负责人•验收•指标
A · MUST 立即 Capture the event, not the corpse 抓住事件，而不是尸体	P: 741 negative rows; 59% of sold product-days ≤0 in the same-day snapshot. Cause: receiving, write-offs and returns are keyed in StoreHub or nowhere; RetailOS only sees polling diffs. Principle: 1+9. Change: phone quick-actions — receive-without-PO, write-off with reason+photo, count straight from a DQ finding (fix links shipped 2026-10-08). 问题：741 行负库存；59% 有销量的商品当日快照 ≤0。根因：收货/报损/退货在 StoreHub 录入或不录，RetailOS 只能看到轮询差异。原则：1+9。改动：手机快捷动作——无单收货、带原因+照片的报损、从 DQ 发现直接发起盘点（修复链接已于 10-08 上线）。	Data: inventory_change + DQ events. Owner: warehouse lead + store managers. Acceptance: a write-off done on a phone leaves actor/reason/photo in the hash-chained audit and the new on-hand; replay gate stays green. KPI: negative rows 741 → <100; % of inventory deltas with a human event attached. 数据：inventory_change+DQ 事件。负责人：仓库主管+门店经理。验收：手机报损在哈希链审计中留下操作人/原因/照片与新库存；重放门槛保持全绿。KPI：负库存 741→<100；带人工事件的库存差异占比。
B · MUST 立即 Receiving truth at the dock 码头收货真相	P: 0 counted receivings ever; 25 stale open POs. Cause: goods receipt is keyed in StoreHub without a physical count. Principle: 1, 9. Change: make the counted-receiving module the default delivery-day flow; Lark nudge with a deep link on PO due date. 问题：点验收货为 0；25 张滞留采购单。根因：收货在 StoreHub 直接录入、未经实物点验。原则：1、9。改动：让点验收货模块成为到货日默认流程；采购单到期当天 Lark 推送+深度链接。	Data: purchase_order + inventory_change. Owner: warehouse lead. Acceptance: PO → counted receiving → variance → issue; the next sync verifies StoreHub. KPI: % of completed POs with a matching receiving record, 0% → >90%. 数据：采购单+库存变动。负责人：仓库主管。验收：采购单→点验收货→差异→工单；下次同步核验 StoreHub。KPI：有收货记录的已完成采购单占比 0%→>90%。
C · MUST 立即 Transfer confirmation loop 调拨确认闭环	P: 0 batches through dispatch/confirm; stock disappears in transit. Cause: the dispatch + store-count confirmation module is unused. Principle: 9. Change: drive adoption of dispatch → store count-confirm; nudges in Lark (deep links shipped 2026-10-08). 问题：发货/确认批次为 0；库存在途丢失。根因：发货+门店点收确认模块闲置。原则：9。改动：推行发货→门店点收确认；Lark 提醒（深度链接已于 10-08 上线）。	Data: order_batch + inventory_change. Owner: warehouse lead + store managers. Acceptance: a dispatched batch cannot close without store confirmation; a difference opens an issue. KPI: % of transfers confirmed; transit differences per week. 数据：调拨批次+库存变动。负责人：仓库主管+门店经理。验收：已发货批次未经门店确认不得关闭；差异自动生成工单。KPI：调拨确认率；每周在途差异数。

Item 项目	Problem → root cause → principle → change 问题→根因→原则→改动	Data • owner • acceptance • KPI 数据•负责人•验收•指标
D · MUST — shipped 已上线 Loud refusals; corrections as candidates 响亮拒绝；修正作为候选	P: ₱51k of needs_review was silent; manager notes were never reused. Cause: reasons were bare codes without a path to the fix. Principle: 3, 6. Change (done 2026-10-08): fix links on /replenishment and /data-quality; decision notes now enter AI context as cited candidate facts. 问题：₱51k 待审查静默无声；经理的备注从未被复用。根因：原因只是裸代码、没有修复入口。原则：3、6。改动（10-08 已上线）：补货与数据质量页的修复链接；经理备注作为候选事实进入 AI 上下文。	Data: recommendation + audit_event. Owner: system (done). Acceptance: every needs_review reason shows a label and a fix link; a manager's note appears in the next AI context. KPI: needs_review ₱51k → 0; % of modify/reject decisions with a written note. 数据：建议+审计事件。负责人：系统（已完成）。验收：每条待审查原因都有标签和修复链接；经理备注出现在下一次 AI 上下文中。KPI：待审查金额 ₱51k→0；修改/拒绝中带文字备注的占比。
E · NEXT 下一步 Replay gate + outcome-linked overrides 重放门槛+结果关联的修正	P: the formula will be tuned on 63 noisy days; overrides are never validated. Cause: no harness existed — replay shipped 2026-10-08 (1,593/1,593 production recommendations reproduce exactly). Principle: 4, 6. Change: extend outcome tracking to compare modified vs accepted outcomes; promote a correction pattern only when it measurably helps. 问题：公式必然要在 63 天带噪声的数据上调优；修改从未被验证。根因：没有验证工具——重放已于 10-08 上线（1,593 条生产建议全部精确复算）。原则：4、6。改动：扩展结果追踪，对比“被修改”与“被接受”的实际结果；只有当修正模式被证明有效时才提升为策略。	Data: decisions + daily snapshots (snapshots began 10-07 — needs runway). Owner: ops manager + system. Acceptance: the override-uplift report refuses to conclude below a minimum sample. KPI: % of overrides with a measured outcome; override uplift vs acceptance. 数据：决策记录+每日快照（快照 10-07 才开始——需要积累）。负责人：运营经理+系统。验收：样本不足时修正效果报告拒绝下结论。KPI：有可测结果的修改占比；修改相对接受的改善幅度。
F · NEXT 下一步 Autonomy scorecard per decision class 按决策类别的自治记分卡	P: autonomy is a single global on/off flag. Cause: amount-only thinking. Principle: 5 (revised). Change: a scorecard per decision class — risk, data confidence, reversibility, historical acceptance rate, amount; shadow mode first, auto-execute only when every threshold holds for 30 days. 问题：自治只是一个全局开关。根因：只看金额的思维。原则：5（修订版）。改动：每个决策类别一张记分卡——风险、数据置信度、可逆性、历史接受率、金额；先影子运行，全部阈值连续 30 天达标才自动执行。	Data: policy-gate logs + acceptance metrics. Owner: management + system. Acceptance: no auto-execution until the class meets all thresholds; every auto action is replayable and audited. KPI: auto-action precision; override rate of auto actions ≈ 0. 数据：策略闸门日志+接受率指标。负责人：管理层+系统。验收：类别未达全部阈值前不得自动执行；每个自动动作可复算、有审计。KPI：自动动作准确率；自动动作被推翻率≈0。
G · NEXT 下一步 Close the loop where people are 在人们常驻的地方闭环	P: 3 missed + 11 pending inspections; expiry/cash modules unused. Cause: alerts had no links; approvals live only inside the app. Principle: 8. Change: Lark alerts now carry deep links (shipped 2026-10-08); next: approve/reject buttons in Lark for low-risk classes within limits, keeping the audit entry. 问题：巡检漏做 3 次、待办 11 次；效期/现金模块闲置。根因：告警没有链接；审批只能在应用内完成。原则：8。改动：Lark 告警已带深度链接（10-08 上线）；下一步：限额内低风险类别在 Lark 里一键批准/拒绝，并保留审计记录。	Data: notifications + audit. Owner: ops manager. Acceptance: a manager approves a low-risk action from Lark and the audit chain shows actor, gate decision and result. KPI: on-time inspection completion %; alert→action time. 数据：通知+审计。负责人：运营经理。验收：经理在 Lark 批准一个低风险动作，审计链完整记录操作人、闸门决策与结果。KPI：巡检按时完成率；告警到处理时长。
H · LATER 以后 Expiry capture + FEFO 效期采集+先到期先出	P: 0 expiry batches recorded. Cause: capture habits do not exist yet — depends on A–C landing first. Principle: 1, 9. Change: expiry capture at receiving (photo + date), FEFO pick order. 问题：效期批次为 0。根因：采集习惯尚未建立——依赖 A–C 先落地。原则：1、9。改动：收货时采集效期（拍照+日期），拣货按先到期先出。	Data: expiry_batch + receiving. Owner: warehouse lead. Acceptance: a received case carries an expiry date; picking lists order by expiry. KPI: expiring value flagged ≥7 days ahead. 数据：效期批次+收货。负责人：仓库主管。验收：收货的每箱都带效期；拣货单按效期排序。KPI：提前 ≥7 天预警的临期金额。
I · LATER 以后 AI reliability transparency AI 可靠性透明化	P: 5 of 20 AI calls failed (404/429/503). Cause: provider chain fragility; failures visible only in logs. Principle: 7. Change: cached last-good brief with its timestamp; public per-provider failure metric on the assistant metrics page. 问题：20 次 AI 调用失败 5 次（404/429/503）。根因：服务商链脆弱；失败只在日志里可见。原则：7。改动：缓存最近一次成功简报并显示时间戳；在助手指标页公开各服务商失败率。	Data: ai_output. Owner: system. Acceptance: with the provider down, the dashboard shows the last good brief with its age and the failure counter. KPI: brief success rate; failed-call rate <5%. 数据：ai_output。负责人：系统。验收：服务商宕机时，看板显示最近一次成功简报及其年龄和失败计数。KPI：简报成功率；失败调用率 <5%。

Source caveat. All company information comes from YC pages, launch posts, company sites and job postings — self-reported marketing by eight very young companies; Haladir's two engineering blog posts are the only substantive technical writing among them. Airgoods' reorder detail comes from a designer's portfolio case study (design intent, not confirmed production). Treat every performance claim as directional. All internal numbers (P1–P6) were queried read-only from the RetailOS production database on 2026-10-08; the “619 phantom stockouts” metric was re-validated and redefined the same day (see the note in Section 1), and the warehouse process map in Section 2 reflects module usage as of that date.

来源说明。所有公司信息均来自 YC 页面、发布帖、官网和招聘帖——即八家非常年轻的公司的自我宣传材料；其中唯一有实质技术含量的公开写作是 Haladir 的两篇工程博客。Airgoods 的补货细节来自一位设计师的作品集案例（属设计意图，非已证实的生产实现）。所有性能数据请视为方向性参考。全部内部数字（P1–P6）均于 2026-10-08 以只读方式查询自 RetailOS 生产数据库；“619 个幽灵缺货”指标已于当日重新校验并重新定义（见第 1 节附注），第 2 节的仓库流程图反映的是截至当日的模块使用情况。